



UNIVERSIDAD DE LOS LLANOS
Facultad de Ciencias básicas e ingenierías

INFORME DE LABORATORIO OPTIMIZACIÓN

Programación lineal – Método simplex estándar y Gran M

Sebastian Obando Rojas ¹

1. Cod: 160004526 Ing. Sistemas
Facultad de Ciencias Básicas e Ingenierías.

Resumen

Este laboratorio implementa el Método Simplex para resolver problemas de Programación Lineal mediante una arquitectura Python modular de tres capas: el núcleo matemático (simplex.py), la interfaz gráfica (interfaz_simplex.py) y el punto de entrada (main.py). La aplicación permite al usuario configurar dinámicamente cualquier número de variables y restricciones, resolver el problema paso a paso visualizando cada iteración de la tabla simplex, e interpretar el resultado óptimo con valores de todas las variables de decisión.

El sistema cubre tanto maximización como minimización. Para la maximización se emplea el simplex estándar con variables de holgura. Para la minimización se implementa el Método de la Gran M, que introduce variables de exceso y variables artificiales penalizadas con un coeficiente M muy grande, permitiendo manejar restricciones de tipo $\leq$, $\geq$ e $=$.

Los conceptos clave implementados incluyen: construcción de la tabla simplex en forma estándar, criterio de optimalidad para detectar la variable entrante, razón mínima para seleccionar la variable saliente, pivoteo de Gauss-Jordan para actualizar la tabla, y la penalización Gran M para forzar la salida de variables artificiales de la base. La interfaz fue desarrollada en CustomTkinter con selector dinámico de tipo de restricción y modo de optimización.

Palabras clave: *Método Simplex, Programación Lineal, Gran M, Variables Artificiales, Optimización, Gauss-Jordan, Python.*

1. Introducción

La Programación Lineal (PL) es una de las herramientas más utilizadas en Investigación de Operaciones para resolver problemas de optimización con restricciones lineales. Desde su formalización por George Dantzig en 1947, el Método Simplex ha sido el algoritmo estándar para resolver este tipo de problemas de manera eficiente, explorando los vértices del conjunto factible hasta encontrar el óptimo.

En entornos reales (logística, manufactura, finanzas, planeación de recursos) los modelos de PL pueden tener decenas de variables y restricciones, lo que hace indispensable contar con herramientas computacionales que automaticen el proceso. Este laboratorio responde a esa necesidad construyendo una solución completa que combina:

- Un motor matemático robusto que implementa fielmente el algoritmo simplex para maximización.
- El Método de la Gran M para minimización con restricciones $\geq$ e $=$, mediante variables artificiales penalizadas.

- Una interfaz gráfica moderna que permite seleccionar el modo (maximizar/minimizar) y el tipo de cada restricción ($\leq$, $\geq$, $=$).
- Una arquitectura limpia y extensible dividida en módulos con responsabilidades claras.

El objetivo es comprender no solo el algoritmo matemático sino también cómo traducirlo en código mantenible, reutilizable y con buena experiencia de usuario, competencias esenciales en ingeniería de sistemas aplicada a ciencias exactas.

2. Referente teórico

Programación lineal: Un problema de Programación Lineal consiste en optimizar (maximizar o minimizar) una función objetivo lineal sujeta a un conjunto de restricciones también lineales y condiciones de no negatividad. La forma estándar de maximización es:

$$\begin{aligned} \text{Maximizar: } z &= c_1x_1 + c_2x_2 + \dots + c_n x_n \\ \text{Sujeto a: } a_{11}x_1 + a_{12}x_2 + \dots + a_{1n} x_n &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n} x_n &\leq b_2 \\ \dots &\leq \dots \\ x_i &\geq 0 \text{ para todo } i \end{aligned}$$

Variables de holgura y forma canónica: Para convertir las desigualdades en igualdades se introduce una variable de holgura $s_i \geq 0$ por cada restricción. Esto lleva el sistema a la forma canónica:

$$\begin{aligned} a_{11}x_1 + \dots + a_{1n}x_n + s_1 &= b_1 \\ a_{21}x_1 + \dots + a_{2n}x_n + s_2 &= b_2 \\ z - c_1x_1 - c_2x_2 - \dots - c_nx_n &= 0 \end{aligned}$$

La base inicial está formada por las variables de holgura, que toman los valores b_i mientras que las variables de decisión valen 0. Este punto corresponde al origen del espacio factible.

Tabla simplex: La tabla simplex organiza el sistema de ecuaciones de manera que permite identificar visualmente las operaciones necesarias. Tiene la forma:

VB		z		x_1	x_2	$\dots$	x_n		s_1	s_2	$\dots$	s_n		CR
s_1		0		a_{11}	a_{12}	$\dots$			1	0	$\dots$			b_1
s_2		0		a_{21}	a_{22}	$\dots$			0	1	$\dots$			b_2
z		1		$-c_1$	$-c_2$	$\dots$			0	0	$\dots$			0

Método de la Gran M (Minimización): Para problemas de minimización con restricciones $\geq$ o $=$, no existe una solución básica factible inicial con solo variables de holgura. El Método de la Gran M resuelve esto introduciendo variables artificiales A_i con un coeficiente de penalización M (un número muy grande) en la función objetivo, de modo que el simplex las expulse automáticamente de la base:

```
Minimizar:  z = c1x1 + c2x2 + ... + cn xn
Sujeto a:   a11x1 + a12x2 + ... + a1n xn ≤ b1
            a21x1 + a22x2 + ... + a2n xn ≤ b2
            ...                               ≤ ...
            xi ≥ 0 para todo i
```

Variables de holgura, artificiales y forma canónica: Para convertir las desigualdades en igualdades se introduce una variable de holgura $s_i \geq 0$ junto con una artificial $A_i \geq 0$ por cada restricción. Esto lleva el sistema a la forma canónica:

```
a11x1 + ... + a1nxn - s1 + A1 = b1
a21x1 + ... + a2nxn - s2 + A2 = b2

z = c1x1 + c2x2 + ... + cn xn + 0s1 + ... 0sn + MA1 + ... MAn → Nueva función Objetivo (Cj)
```

Tabla Gran M: La tabla Gran M organiza el sistema de ecuaciones de manera que permite identificar visualmente las operaciones necesarias. Tiene la forma:

```
Cj | c1x1 + c2x2 + ... + cn xn + 0s1 + ... 0sn + MA1 + ... MAn
VB | x1  x2  ... xn | s1  s2  ... sn | CR
-----
M1 A1 | a11 a12 ...   | 1   0   ...   | b1
M2 A2 | a21 a22 ...   | 0   1   ...   | b2
-----
Zj | [(M1*a11) + (M2*a21)] así para cada columna donde se encuentran las filas de las
variables artificiales
Cj - Zj |
```

Debemos tener en cuenta que para que las iteraciones terminen, $C_j - Z_j$ ya no deben tener números negativos.

Arquitectura del software:

Concepto	Definición breve	Aplicación en el Lab
Python 3	Lenguaje de propósito general con tipado dinámico	Núcleo del algoritmo y lógica de la GUI
CustomTkinter	Biblioteca de widgets modernos sobre Tkinter	Construcción de la interfaz gráfica
Fractions	Aritmética exacta de fracciones racionales (stdlib)	Conversión de decimales a fracciones en la tabla de resultados
copy.deepcopy	Copia profunda de estructuras de datos	Guardar historial de tablas sin referencias compartidas
typing	Anotaciones de tipo estáticas	Documentación y legibilidad del código (List, Optional, Tuple)
Gran M	Penalización M muy grande en variables artificiales	Permite minimizar con restricciones $\geq e$ = sin solución básica inicial

3. Procedimiento

3.1 Construcción de la Tabla Simplex Inicial

La función `construir_tabla_inicial` recibe los coeficientes c (función objetivo), la matriz A (restricciones) y el vector b (términos independientes). Agrega variables de holgura y devuelve la tabla en forma canónica.

Archivo: *simplex.py*

```
def construir_tabla_inicial(c, A, b):
    n = len(c)    # variables de decisión
    m = len(b)    # restricciones
    nombres_var = [f'x{i+1}' for i in range(n)] + \
                  [f's{i+1}' for i in range(m)]
    base = [f's{i+1}' for i in range(m)] # base inicial

    # Fila z: coeficientes -c para vars de decisión, 0 para holguras, CR=0
    fila_z = [-c[j] for j in range(n)] + [0.0] * m + [0.0]
    tabla = [fila_z]

    # Filas de restricciones con identidad para holguras
    for i in range(m):
        fila = list(A[i]) + \
              [1.0 if j == i else 0.0 for j in range(m)] + \
              [b[i]]
        tabla.append(fila)

    return tabla, nombres_var, base
```

3.2 Criterio de Optimalidad — Variable Entrante

La función encontrar_variable_entrante recorre la fila z buscando el coeficiente más negativo entre las variables de decisión. Si no existe ninguno negativo, el problema ya es óptimo.

Archivo: simplex.py

```
def encontrar_variable_entrante(tabla, n, m):
    fila_z = tabla[0]
    min_val = 0.0
    j_entrante = None
    for j in range(n):          # Solo variables de decisión
        if fila_z[j] < min_val: # Busca el más negativo
            min_val = fila_z[j]
            j_entrante = j
    return j_entrante          # None => solución óptima
```

3.3 Criterio de Razón Mínima — Variable Saliente

Para determinar qué variable sale de la base se aplica el criterio de razón mínima: se divide el término independiente CR entre el coeficiente de la columna entrante, pero solo cuando dicho coeficiente es positivo.

Archivo: simplex.py

```
def encontrar_variable_saliente(tabla, j_entrante, m):
    i_saliente = None
    min_ratio = float('inf')
    for i in range(1, m + 1):      # Filas de restricciones
        coef = tabla[i][j_entrante]
        if coef <= 0:              # Ignorar no positivos
            continue
        ratio = tabla[i][-1] / coef # CR / coeficiente
        if ratio < min_ratio:
            min_ratio = ratio
            i_saliente = i
    return i_saliente             # None => no acotado
```

3.4 Pivoteo de Gauss-Jordan

El pivoteo normaliza la fila pivote dividiéndola por el elemento pivote, luego hace cero todos los demás elementos en esa columna usando eliminación de Gauss-Jordan. Modifica la tabla in-place.

Archivo: *simplex.py*

```
def hacer_pivoteo(tabla, i_pivote, j_pivote):
    ep = tabla[i_pivote][j_pivote]
    if abs(ep) < 1e-12:
        raise ValueError('Pivote numéricamente cero.')

    # 1. Normalizar fila pivote (divide por ep)
    for j in range(len(tabla[0])):
        tabla[i_pivote][j] /= ep

    # 2. Hacer ceros en toda la columna pivote
    for i in range(len(tabla)):
        if i == i_pivote:
            continue
        factor = tabla[i][j_pivote]
        for j in range(len(tabla[0])):
            tabla[i][j] -= factor * tabla[i_pivote][j]
```

3.5 Ciclo Principal del Simplex

La función ejecutar_simplex coordina todo el algoritmo, manteniendo un historial de tablas (copias profundas) para mostrarlo en la GUI iteración por iteración.

Archivo: *simplex.py*

```
def ejecutar_simplex(c, A, b, max_iter=100):
    n, m = len(c), len(b)
    tabla, nombres_var, base = construir_tabla_inicial(c, A, b)
    historial = []

    for _ in range(max_iter):
        j_entrante = encontrar_variable_entrante(tabla, n, m)

        if j_entrante is None: # Criterio de optimalidad satisfecho
            historial.append((copy.deepcopy(tabla), ...None, None, None))
            # Extraer solución de la tabla
            sol = {f'x{i+1}': 0.0 for i in range(n)}
            sol['z'] = tabla[0][-1]
            for i, vb in enumerate(base):
                if vb.startswith('x'):
                    sol[vb] = tabla[i + 1][-1]
            return historial, {'optimo': True, 'solucion': sol}

        i_saliente = encontrar_variable_saliente(tabla, j_entrante, m)
        if i_saliente is None: # No acotado
            return historial, {'optimo': False, 'no_acotado': True}

        ep = tabla[i_saliente][j_entrante]
        historial.append((copy.deepcopy(tabla), list(nombres_var),
                        list(base), j_entrante, i_saliente, ep))
        base[i_saliente - 1] = nombres_var[j_entrante] # Actualizar base
        hacer_pivoteo(tabla, i_saliente, j_entrante)

    return historial, {'optimo': False, 'max_iter': True}
```

3.6 Construcción de la Tabla Inicial — Gran M

Se añadió la función `construir_tabla_gran_m` para soportar minimización con los tres tipos de restricción. Según el tipo, asigna las variables auxiliares correspondientes y construye la fila z con la penalización M. Luego, elimina las artificiales básicas de la fila z mediante operaciones de fila, dejando la tabla lista para iterar.

Archivo: simplex.py

```
def construir_tabla_gran_m(c, A, b, tipos, M=1e6):

    fila_z = coef_z + [0.0] # CR = 0
    # Eliminar artificiales básicas de la fila z:
    for i in range(m):
        if col_artificial[i] is not None:
            for j in range(total_cols + 1):
                fila_z[j] -= M * filas_rest[i][j]
    tabla = [fila_z] + filas_rest
    return tabla, nombres_var, base
```

3.7 Ciclo Principal — Minimización Gran M

La función `ejecutar_simplex_minimizacion` reutiliza el mismo criterio de entrada (coeficiente más negativo en la fila z) pero considerando todas las columnas, incluyendo excesos y artificiales. Al converger, verifica que ninguna artificial permanezca en la base: si alguna persiste con valor positivo, el problema es infactible. El valor de z se reconstruye con los coeficientes originales c, sin la penalización M.

Archivo: simplex.py

```
def ejecutar_simplex_minimizacion(c, A, b, tipos=None, M=1e6):
    if tipos is None: tipos = ['<='] * len(b)
    tabla, nombres_var, base = construir_tabla_gran_m(c,A,b,tipos,M)
    n_total = len(nombres_var)
    for _ in range(max_iter):
        # Coef más negativo en fila z (todas las columnas)
        fila_z = tabla[0]
        j_entrante = None
        for j in range(n_total):
            if fila_z[j] < -1e-8:
                if j_entrante is None or fila_z[j] < fila_z[j_entrante]:
                    j_entrante = j
        if j_entrante is None:
            # Verificar que no quede artificial en la base
            if any(v.startswith('A') for v in base):
                return historial, {'optimo': False, 'infactible': True}
            # Recalcular z real sin penalización M
            z_real = sum(c[int(vb[1:])-1] * tabla[i+1][-1]
```

```

        for i, vb in enumerate(base) if vb.startswith('x'))

    sol['z'] = z_real
    return historial, {'optimo': True, 'solucion': sol,
                      'minimizacion': True}

# Mismo pivoteo que maximización
i_saliente = encontrar_variable_saliente(tabla, j_entrante, m)
hacer_pivoteo(tabla, i_saliente, j_entrante)
return historial, {'optimo': False, 'max_iter': True}

```

3.8 Interfaz Gráfica

La interfaz contiene dos elementos: un botón que alterna entre MAXIMIZAR y MINIMIZAR (azul para max, morado para min), y un selector de tipo de restricción ($\leq$, $\geq$, $=$) que aparece junto al campo b de cada restricción al generar el formulario. Al resolver, se leen los tipos seleccionados y se invocan los motores correspondientes.

Archivo: *interfaz_simplex.py*

```

# Botón para alternar modo
self.btn_mod0 = ctk.CTkButton(
    obj_label_frame, text='▲ MAXIMIZAR',
    command=self.toggle_mod0, fg_color='#1565C0')

def toggle_mod0(self):
    self.mod0_minimizar = not self.mod0_minimizar
    if self.mod0_minimizar:
        self.btn_mod0.configure(text='▼ MINIMIZAR', fg_color='#7B1FA2')
    else:
        self.btn_mod0.configure(text='▲ MAXIMIZAR', fg_color='#1565C0')

# Selector por restricción (nuevo, dentro de generar_campos_restricciones)
tipo_menu = ctk.CTkOptionMenu(frame, values=['<=', '>=', '='])
tipo_menu.set('<=')

# Al resolver, se pasa el tipo al motor correspondiente
tipos = [m.get() for m in self.entries_tipos]
if self.mod0_minimizar:
    historial, resultado = ejecutar_simplex_minimizacion(c, A, b, tipos)
else:
    historial, resultado = ejecutar_simplex(c, A, b)

```

4. Resultados

4.1 Ejemplo de Ejecución

Para ilustrar el funcionamiento se resuelve el siguiente problema mediante Maximización y Minimización:

$$\begin{aligned} z &= 3x_1 + 5x_2 \\ \text{Sujeto a: } x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 5x_2 &\leq 25 \\ x_1, x_2 &\geq 0 \end{aligned}$$

4.2 Visualización en la interfaz

Ingresamos los datos en la interfaz del programa para proceder a ejecutar:

The figure displays two screenshots of a software interface for configuring a linear programming problem. The left screenshot shows the 'Configurar Problema' window with 'Nº variables' set to 2 and 'Nº restricciones' set to 3. The 'FUNCIÓN OBJETIVO' section shows the formula $z = [] \cdot x_1 + [] \cdot x_2$ with coefficients $x_1: 3$ and $x_2: 5$. The 'RESTRICCIONES' section shows three constraints: Restricción 1 ($x_1: 1, x_2: 0, b: 4$), Restricción 2 ($x_1: 0, x_2: 2, b: 12$), and Restricción 3 ($x_1: 3, x_2: 5, b: 25$). The right screenshot shows the same window but with the 'MINIMIZAR' button selected instead of 'MAXIMIZAR'.

Figura.1. Se aprecia la configuración del problema donde se agrega el número de variables, el número de restricciones y se crea el formulario, luego se introducen los valores, primeramente, en la función objetivo y luego en el apartado de restricciones, luego tenemos las dos opciones tanto para max o min.

4.3 Iteraciones

La tabla inicial muestra para cada opción tanto max como min, la estructura dictada por el profesor.

ITERACIONES Y RESULTADO

MÉTODO SIMPLEX - MAXIMIZACIÓN

Iteración 1

VB	z	x1	x2	s1	s2	s3	CR	
s1	0	1	0	1	0	0	4	
s2	0	0	2	0	1	0	12	
s3	0	3	5	0	0	1	25	← fila pivote
z	1	-3	-5	0	0	0	0	

Variable entrante (columna pivote): x2

Variable saliente (fila pivote): s3

Elemento pivote: 5

Iteración 2

VB	z	x1	x2	s1	s2	s3	CR	
s1	0	1	0	1	0	0	4	
s2	0	-6/5	0	0	1	-2/5	2	
x2	0	3/5	1	0	0	1/5	5	
z	1	0	0	0	0	1	25	

ITERACIONES Y RESULTADO

MÉTODO SIMPLEX - MINIMIZACIÓN (Gran M)

Iteración 1

Cj	3	5	0	M	0	M	0	M	
VB	x1	x2	s1	A1	S2	A2	S3	A3	CR
A1	1	0	-1	1	0	0	0	0	4
A2	0	2	0	0	-1	1	0	0	12
A3	3	5	0	0	0	0	-1	1	25
zj	4M	7M	-M	M	-M	M	-M	M	41M
Cj-zj	-3999997	-6999995	M	0	M	0	M	0	

Entrante: x2

Saliente: A3

Pivote: 5

Iteración 2

Cj	3	5	0	M	0	M	0	M	
VB	x1	x2	s1	A1	S2	A2	S3	A3	CR
A1	1	0	-1	1	0	0	0	0	4
A2	-6/5	0	0	0	-1	1	2/5	-2/5	2
x2	3/5	1	0	0	0	0	-1/5	1/5	5
zj	-199997	5	-M	M	-M	M	399999	-399999	6000025
Cj-zj	200000	0	M	0	M	0	-399999	1399999	

Entrante: S3

Saliente: A2

Pivote: 2/5

Figura.2. Al momento de haber ingresado los debidos datos como se muestra en la figura 1, se oprime en la opción “Resolver”, dándonos las iteraciones necesarias para hallar la solución.

4.3 Resultado Final

Tras las iteraciones necesarias, el algoritmo converge a la solución óptima. La GUI muestra el resultado

claramente diferenciado del resto de las iteraciones:

RESULTADO FINAL	RESULTADO FINAL
Valor óptimo: $z_{\max} = 25$ Variables de decisión: $x_1 = 0$ $x_2 = 5$	Valor óptimo: $z_{\min} = 42$ Variables de decisión: $x_1 = 4$ $x_2 = 6$

Figura.3. Después de las iteraciones necesarias hasta que los datos de z para maximización y $C_j - Z_j$ para minimización lleguen a ser positivos, nos muestra el resultado final.

Los valores se presentan como fracciones exactas gracias al módulo Fraction, lo que facilita la verificación manual de los resultados.

5. Conclusiones

Este laboratorio permitió consolidar tanto el conocimiento matemático del Método Simplex como las competencias de desarrollo de sistemas necesarias para implementarlo y extenderlo.

El Método Simplex es elegante en su simplicidad conceptual: cada iteración mejora o mantiene el valor de z moviéndose a un vértice adyacente del poliedro factible.

La separación entre los criterios de optimalidad (variable entrante) y factibilidad (variable saliente) es clave para la correcta convergencia del algoritmo.

El pivoteo de Gauss-Jordan garantiza que la nueva variable básica tenga un '1' en su columna y '0' en todas las demás filas, manteniendo la identidad en la base.

El Método de la Gran M es una extensión natural del simplex: reemplaza la necesidad de una solución básica factible inicial con variables artificiales fuertemente penalizadas que el propio algoritmo expulsa. El criterio de entrada es idéntico al de maximización cuando la tabla se construye y ajusta correctamente.

La construcción de la fila z en minimización requiere un paso adicional: eliminar las artificiales básicas operando sobre la fila z antes de comenzar las iteraciones, para que los coeficientes reflejen el costo real sin duplicar la penalización M .

El historial de tablas es una herramienta didáctica invaluable para entender cómo evoluciona la solución iteración a iteración, y para verificar que las variables artificiales efectivamente abandonan la base.

6. Bibliografía

[1] Universidad de los Llanos, Guía 3: Programación Lineal – Método Simplex,” Laboratorio de Optimización, Ingeniería de Sistemas, 2023